PROJETO FINAL · LABORATÓRIO DE BANCO DE DADOS

CineControl

Relatório Final — Etapa 1 revisada e Etapa 2: Operação, Interface e Desempenho

Disciplina	**Laboratório de Banco de Dados — 2026/2 (UCB, Engenharia de Software)**
Base documental	**Documento de Visão, HU01, roteiro da disciplina e Projeto Final**
SGBD	**MySQL 8.0+**
Aplicação	**Python 3.10+ · Streamlit · mysql-connector-python**
Entregáveis Etapa 1	**01_ddl.sql · 02_carga.sql · 03_consultas.sql**
Entregáveis Etapa 2	**04_etapa2.sql · 05_otimizacao.sql · seed_etapa2.sql · app.py · db.py**
Autores no repositório	**André · Victor Emanoel · Paulo dos Santos · Igor · *Iasmin***
Entrega	**Etapa 1: 20/09/2026 · Etapa 2: 22/11/2026 (AVA)**

Decisão de modelagem.

Os documentos do CineControl descrevem um domínio inicialmente mais simples. Para atender aos requisitos mínimos da Etapa 1, o modelo foi ampliado com gênero, ator/elenco, assento, histórico temporal e especialização de usuário, mantendo o núcleo funcional de cinema. Na Etapa 2 o esquema permaneceu o mesmo e foram acrescentados apenas objetos de operação: views, tabela e gatilhos de auditoria, índices adicionais e perfis de acesso.

Sumário

Parte	Conteúdo	Artefatos
I — Etapa 1	A1 Escopo e regras · A2 MER · A3 Dicionário · A4 Modelo lógico · A5 Normalização · A6–A8 Implementação, carga e consultas	01_ddl.sql · 02_carga.sql · 03_consultas.sql
II — Etapa 2	B1 Aplicação · B2 Acesso a dados · B3 Transações e concorrência · B4 Otimização · B5 Segurança · B6 Auditoria · B7 Projeto físico	04_etapa2.sql · 05_otimizacao.sql · app.py · db.py
III — Fechamento	B8 Revisão da Etapa 1 · Declaração de uso de IA · Reprodução do ambiente · Checklist da Etapa 2	README.md

PARTE I — Etapa 1: Modelagem, Implementação e Consultas

A1 — Escopo e regras de negócio

O CineControl é um sistema web para gerenciamento das atividades básicas de um cinema, incluindo usuários, filmes, salas, sessões e reservas. O sistema possui interface, API e banco de dados; administradores gerenciam cadastros e clientes consultam a programação e realizam reservas.

O modelo mantém fora do escopo pagamentos reais, integração com cartão/Pix, nota fiscal, sistemas externos de cinema, SMS, aplicativo mobile, fidelidade, promoções complexas, financeiro e lanchonete.

A coluna **Implementação** é a rastreabilidade exigida entre cada regra e o objeto que a faz valer. Nas regras atendidas pela aplicação, a coluna indica o ponto do código da Etapa 2 em que a regra é exercida.

ID	Regra verificável	Atendimento	Implementação
RN01	O usuário deve informar credenciais válidas para acessar o sistema.	Aplicação	usuario.senha_hash (hash + sal); no protótipo Streamlit não há tela de login — o acesso é controlado pelos perfis de banco (B5).
RN02	Funcionalidades administrativas devem estar disponíveis somente a usuários com perfil de administrador.	Aplicação	Tabela administrador; perfis cc_leitura, cc_operacional e cc_gestor com GRANT mínimo (B5).
RN03	Todo filme deve possuir título, duração e classificação válidos.	Banco	NOT NULL; ck_filme_duracao (1–500); ck_filme_classificacao.
RN04	Toda sala deve possuir capacidade positiva.	Banco	ck_sala_capacidade (1–1000).
RN05	Toda sessão deve estar associada a exatamente um filme e uma sala.	Banco	id_filme e id_sala NOT NULL; fk_sessao_filme; fk_sessao_sala.
RN06	Uma sala não pode possuir duas sessões no mesmo instante.	Banco	uq_sessao_sala_inicio (id_sala, inicio).
RN07	Uma reserva confirmada somente pode ser criada quando houver ingressos disponíveis.	Aplicação / Transação	Reserva + UPDATE em uma transação (app.py); ck_sessao_disponibilidade impede saldo negativo.

ID	Regra verificável	Atendimento	Implementação
RN08	A quantidade de ingressos confirmados de uma sessão não pode ultrapassar a capacidade da sala.	Aplicação / Consulta	Consulta 13 (ocupação) e view vw_relatorio_sessoes; contador ingressos_disponiveis.
RN09	Um cliente somente pode cancelar uma reserva registrada em seu nome.	Aplicação	fk_reserva_cliente vincula a reserva ao titular; cancelamento transacional na aba Reservas.
RN10	Registros dependentes devem ser impedidos de excluir quando a exclusão quebrar a integridade do domínio.	Banco	ON DELETE RESTRICT em fk_reserva_*, fk_sessao_*, fk_filme_origem; erro tratado na interface.
RN11	Relacionamentos entre filme, sala, sessão e reserva devem permanecer consistentes.	Banco	Chaves estrangeiras com ON UPDATE CASCADE.
RN12	Inclusões, alterações e exclusões devem refletir corretamente no banco.	Banco	InnoDB transacional; CRUD da Etapa 2 com commit/rollback explícitos.
RN13	Cada e-mail de usuário deve ser único.	Banco	uq_usuario_email.
RN14	Uma associação filme–gênero não pode ser repetida para o mesmo par.	Banco	pk_filme_genero (id_filme, id_genero).
RN15	Cada filme pode ter no máximo um gênero marcado como principal.	Banco	Gatilhos trg_filme_genero_principal_ins/upd (MySQL não tem índice único parcial).
RN16	Um filme pode possuir vários atores e cada participação deve registrar personagem e ordem de crédito.	Modelo / Associação	Tabela filme_ator; personagem NOT NULL; ck_filme_ator_ordem.
RN17	Uma sala pode possuir vários assentos, e cada assento é identificado pelo par sala+número.	Banco	pk_assento (id_sala, numero) — entidade fraca.
RN18	Um filme pode ser continuação de outro filme; a referência deve apontar para um filme existente.	Banco	fk_filme_origem (autorrelacionamento, nulável).
RN19	O histórico de situação do filme deve registrar início e, quando encerrado, fim posterior ao início.	Banco	ck_historico_periodo (fim IS NULL OR fim > inicio).
RN20	As reservas devem ser realizadas por clientes cadastrados e vinculadas a sessões existentes.	Banco	fk_reserva_cliente → cliente; fk_reserva_sessao → sessao.

A2 — MER conceitual

Notação de Engenharia da Informação (pé de galinha), mantida do início ao fim. Especialização USUARIO → CLIENTE/ADMINISTRADOR: total e exclusiva. Relacionamentos N:N entre FILME–GÊNERO e FILME–ATOR, ambos com atributos próprios (*principal*; *personagem* e *ordem_credito*). ASSENTO é entidade fraca identificada por sala+número. FILME possui autorrelacionamento de continuação. HISTORICO_SITUACAO_FILME registra a evolução temporal da situação do filme.

Requisitos mínimos de complexidade (item 4 do roteiro)

Requisito	Mínimo	Atendimento no CineControl
Entidades	8 (sem associativas)	11: USUARIO, CLIENTE, ADMINISTRADOR, FILME, GENERO, ATOR, SALA, ASSENTO, SESSAO, RESERVA, HISTORICO_SITUACAO_FILME
Relacionamentos N:N	2 com atributo próprio	FILME–GENERO (principal) e FILME–ATOR (personagem, ordem_credito)
Autorrelacionamento	1	FILME → FILME (continuação, id_filme_origem)
Generalização/especialização	1, com mapeamento justificado	USUARIO → CLIENTE / ADMINISTRADOR (total e exclusiva)
Entidade fraca	1	ASSENTO (identificada por id_sala + numero)
Atributo temporal	1 entidade com histórico	HISTORICO_SITUACAO_FILME (inicio, fim)
Regras de negócio	20	RN01–RN20 (A1)
Volume de carga	40 nas principais / 100 na maior	RESERVA com 200 registros (carga da Etapa 1)

A Figura 1 apresenta o diagrama completo.

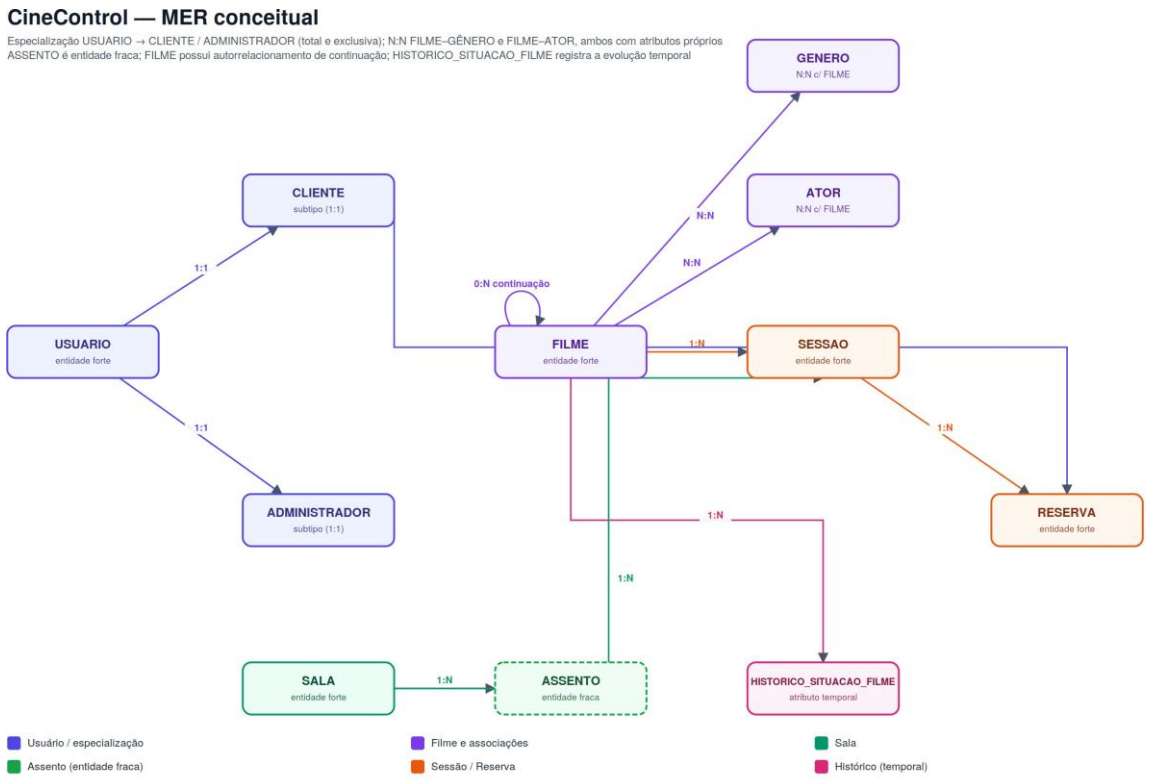


Figura 1 — MER conceitual do CineControl.

A3 — Dicionário de dados conceitual

Versão condensada, derivada do script físico. O documento completo está em `docs/dicionario-dados.pdf`. PK = chave primária; FK = chave estrangeira; UQ = único; Obrig. = obrigatoriedade.

USUARIO

Atributo	Descrição	Domínio	Obrig.	Observação
id_usuario	Identificador interno	BIGINT sequencial	Sim	PK substituta
nome	Nome completo	Texto até 120	Sim	
email	E-mail de acesso	Texto até 100	Sim	RN13 — UQ
senha_hash	Hash da senha	Texto até 255	Sim	RN01 — nunca texto claro
ativo	Conta ativa	Booleano, padrão verdadeiro	Sim	
criado_em	Data de criação	Timestamp, padrão atual	Sim	

CLIENTE

Atributo	Descrição	Domínio	Obrig.	Observação
id_usuario	Usuário especializado	BIGINT	Sim	PK e FK → usuario (CASCADE)
data_cadastro	Data de cadastro	Data, padrão hoje	Sim	

ADMINISTRADOR

Atributo	Descrição	Domínio	Obrig.	Observação
id_usuario	Usuário especializado	BIGINT	Sim	PK e FK → usuario (CASCADE)
nivel_acesso	Nível do administrador	OPERACIONAL | GESTOR	Sim	RN02 — ck_administrador_nivel

FILME

Atributo	Descrição	Domínio	Obrig.	Observação
id_filme	Identificador	BIGINT sequencial	Sim	PK substituta
id_filme_origem	Filme anterior da franquia	BIGINT	Não	RN18 — FK autorreferente
titulo	Título	Texto até 180	Sim	RN03
descricao	Sinopse	Texto longo	Não	Aceita ausência
duracao_min	Duração em minutos	Inteiro 1–500	Sim	RN03
classificacao	Classificação indicativa	L, 10, 12, 14, 16, 18	Sim	RN03
data_lancamento	Lançamento	Data	Não	Filmes futuros podem não ter data
situacao	Situação corrente	EM_CARTAZ, PAUSADO, ENCERRADO, FUTURO	Sim	Espelha o último evento do histórico

GENERO

Atributo	Descrição	Domínio	Obrig.	Observação
id_genero	Identificador	BIGINT	Sim	PK
nome	Nome do gênero	Texto até 60	Sim	UQ

ATOR

Atributo	Descrição	Domínio	Obrig.	Observação
id_ator	Identificador	BIGINT	Sim	PK
nome	Nome artístico (fictício)	Texto até 120	Sim	
data_nascimento	Nascimento	Data	Não	Aceita ausência

FILME_GENERO

Atributo	Descrição	Domínio	Obrig.	Observação
id_filme, id_genero	Par filme–gênero	BIGINT, BIGINT	Sim	RN14 — PK composta
principal	Gênero principal	Booleano	Sim	RN15 — um por filme

FILME_ATOR

Atributo	Descrição	Domínio	Obrig.	Observação
id_filme, id_ator	Par filme–ator	BIGINT, BIGINT	Sim	PK composta
personagem	Personagem	Texto até 120	Sim	RN16
ordem_credito	Ordem nos créditos	SMALLINT > 0	Sim	RN16

SALA

Atributo	Descrição	Domínio	Obrig.	Observação
id_sala	Identificador	BIGINT	Sim	PK
nome_numero	Nome ou número	Texto até 40	Sim	UQ
capacidade	Lugares	Inteiro 1–1000	Sim	RN04
situacao	Situação	ATIVA, MANUTENCAO, INATIVA	Sim	

ASSENTO

Atributo	Descrição	Domínio	Obrig.	Observação
id_sala, numero	Identificação do assento	BIGINT, SMALLINT > 0	Sim	RN17 — PK composta (entidade fraca)
fileira	Fileira	1 letra A–Z	Sim	ck_assento_fileira
ativo	Disponível para uso	Booleano	Sim	

SESSAO

Atributo	Descrição	Domínio	Obrig.	Observação
id_sessao	Identificador	BIGINT	Sim	PK
id_filme, id_sala	Filme e sala	BIGINT	Sim	RN05 — FKs
inicio	Início da exibição	Timestamp	Sim	RN06 — UQ com id_sala
preco	Preço do ingresso	NUMERIC(8,2) ≥ 0	Sim	
ingressos_disponiveis	Saldo de ingressos	Inteiro ≥ 0	Sim	RN07/RN08
situacao	Situação	PROGRAMADA, EM_EXIBICAO, ENCERRADA, CANCELADA	Sim	

RESERVA

Atributo	Descrição	Domínio	Obrig.	Observação
id_reserva	Identificador	BIGINT	Sim	PK
id_cliente	Titular	BIGINT	Sim	RN20 — FK → cliente
id_sessao	Sessão	BIGINT	Sim	RN20 — FK → sessao
quantidade	Ingressos	SMALLINT > 0	Sim	
data_reserva	Momento da reserva	Timestamp, padrão atual	Sim	
situacao	Situação	CONFIRMADA, CANCELADA	Sim	

HISTORICO_SITUACAO_FILME

Atributo	Descrição	Domínio	Obrig.	Observação
id_filme, sequencia	Evento do histórico	BIGINT, INTEGER	Sim	PK composta
situacao	Situação no período	Mesmo domínio de filme.situacao	Sim	
inicio	Início do período	Timestamp	Sim	
fim	Fim do período	Timestamp	Não	RN19 — nulo = em aberto

AUDITORIA_RESERVA (Etapa 2)

Atributo	Descrição	Domínio	Obrig.	Observação
id_auditoria	Identificador	BIGINT	Sim	PK
operacao	Tipo da operação	INSERT | UPDATE	Sim	B6
id_reserva	Reserva afetada	BIGINT	Não	Sem FK, para preservar o rastro após exclusões
autor	Conta do banco que alterou	Texto até 100, padrão USER()	Não	B6
momento	Data e hora	Timestamp	Sim	B6
valor_ant / valor_novo	Valores antes e depois	JSON	Não	Anterior é nulo no INSERT

A4 — Modelo lógico relacional

O modelo lógico completo, com todas as relações, chaves primárias e estrangeiras, está em `docs/modelo-logico.pdf` e reproduzido na página seguinte. Esquema (chaves primárias em **negrito sublinhado**, estrangeiras em *itálico*):

Relação	Atributos
USUARIO	**id_usuario**, nome, email (UQ), senha_hash, ativo, criado_em
CLIENTE	***id_usuario***, data_cadastro
ADMINISTRADOR	***id_usuario***, nivel_acesso
FILME	**id_filme**, *id_filme_origem*, titulo, descricao, duracao_min, classificacao, data_lancamento, situacao
GENERO	**id_genero**, nome (UQ)
ATOR	**id_ator**, nome, data_nascimento
FILME_GENERO	***id_filme***, ***id_genero***, principal
FILME_ATOR	***id_filme***, ***id_ator***, personagem, ordem_credito

Relação	Atributos
SALA	**id_sala**, nome_numero (UQ), capacidade, situacao
ASSENTO	***id_sala***, **numero**, fileira, ativo
SESSAO	**id_sessao**, *id_filme*, *id_sala*, inicio, preco, ingressos_disponiveis, situacao — UQ(id_sala, inicio)
RESERVA	**id_reserva**, *id_cliente*, *id_sessao*, quantidade, data_reserva, situacao
HISTORICO_SITUACAO_FILME	***id_filme***, **sequencia**, situacao, inicio, fim

Decisões de mapeamento e justificativas

- **Especialização USUARIO → CLIENTE/ADMINISTRADOR: três tabelas (supertipo + dois subtipos 1:1).** Os subtipos têm atributos próprios e disjuntos (data_cadastro × nivel_acesso); uma tabela única geraria colunas quase sempre nulas, e uma tabela por subclasse duplicaria nome, e-mail e senha e impediria a unicidade global do e-mail (RN13) e uma FK única para o usuário. A chave dos subtipos é ao mesmo tempo PK e FK (CASCADE). O MySQL não possui restrição declarativa para totalidade e exclusividade; ambas ficam sob responsabilidade da aplicação.
- **Relacionamentos N:N → tabelas associativas.** FILME_GENERO e FILME_ATOR têm PK composta pelas duas FKs, o que impede duplicidade de par (RN14) e acomoda os atributos da associação. Exclusão de filme remove suas associações (CASCADE); gênero e ator em uso não podem ser excluídos (RESTRICT).
- **Entidade fraca ASSENTO.** Chave composta (id_sala, numero) com ON DELETE CASCADE: um assento não existe sem sua sala. Surrogate key seria redundante, pois o par já identifica o assento (RN17).
- **Autorrelacionamento de FILME.** Coluna nulável id_filme_origem com RESTRICT: filmes sem origem são o caso comum, e impedir a exclusão do filme original preserva a cadeia de continuação.
- **Histórico temporal.** Entidade dependente com PK (id_filme, sequencia); fim nulo representa o período em aberto, e ck_historico_periodo garante fim > inicio (RN19).
- **Chave natural × substituta.** Entidades principais usam BIGINT AUTO_INCREMENT, pois seus identificadores naturais (e-mail, nome da sala, título) são mutáveis ou longos. E-mail, nome_numero (sala) e nome (gênero) permanecem como chaves candidatas protegidas por UNIQUE. Associativas e entidades dependentes usam chaves naturais compostas, porque a identidade vem das relações.
- **Ações referenciais.** CASCADE onde o dependente não faz sentido sozinho (subtipos, assentos, associações, histórico); RESTRICT onde a exclusão destruiria registros de negócio (reservas, sessões, gêneros, atores, filme de origem); ON UPDATE CASCADE em todas as FKs.

CineControl — Modelo Lógico Relacional

Banco: cinecontrol | SGBD: MySQL 8.0+ | 13 tabelas | Derivado do script qsl/01_ddl.sql (Etapa 1 — Laboratório de Banco de Dados)
Notação: cada retângulo é uma tabela; PK = chave primária, FK = chave estrangeira, UQ = restrição UNIQUE; 1 / N = cardinalidade do relacionamento.

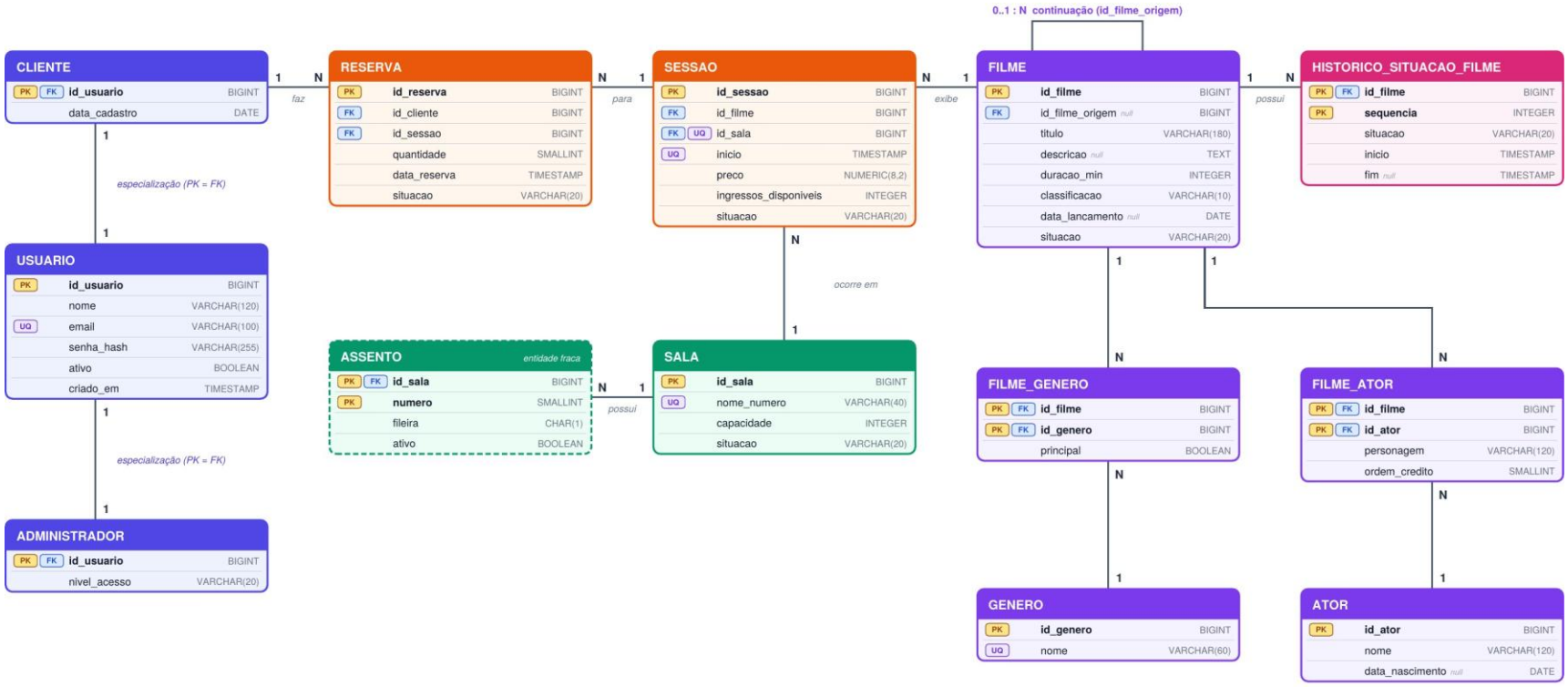


Figura 2 — Modelo lógico relacional (docs/modelo-logico.pdf, página 1).

A5 — Verificação de normalização

O esquema foi projetado para a Terceira Forma Normal (3FN) e verificado relação a relação. Todas as relações estão também na Forma Normal de Boyce-Codd (BCNF): em cada uma, todo determinante é chave candidata. As duas redundâncias controladas estão descritas ao final desta seção.

Relação	Dependências funcionais relevantes	Justificativa
USUARIO	id_usuario → nome, email, senha_hash, ativo, criado_em; email → id_usuario	Atributos dependem da chave; e-mail é candidato único. BCNF.
CLIENTE	id_usuario → data_cadastro	Chave simples. BCNF.
ADMINISTRADOR	id_usuario → nivel_acesso	Chave simples. BCNF.
FILME	id_filme → id_filme_origem, titulo, descricao, duracao_min, classificacao, data_lancamento, situacao	Sem dependência transitiva. Ver redundância R1.
FILME_GENERO	(id_filme, id_genero) → principal	Depende da chave completa. BCNF.
FILME_ATOR	(id_filme, id_ator) → personagem, ordem_credito	Depende da chave completa. BCNF.
SALA	id_sala → nome_numero, capacidade, situacao; nome_numero → id_sala	Chave substituta e candidata. BCNF.
ASSENTO	(id_sala, numero) → fileira, ativo	Sem dependência parcial. BCNF.
SESSAO	id_sessao → id_filme, id_sala, inicio, preco, ingressos_disponiveis, situacao; (id_sala, inicio) → id_sessao	Atributos dependem da chave; (id_sala, inicio) é candidata (RN06). Ver redundância R2.
RESERVA	id_reserva → id_cliente, id_sessao, quantidade, data_reserva, situacao	Atributos dependem da chave. BCNF.
HISTORICO_SITUACAO_FILME	(id_filme, sequencia) → situacao, inicio, fim	Evento identificado pela chave composta. BCNF.
AUDITORIA_RESERVA	id_auditoria → operacao, id_reserva, autor, momento, valor_ant, valor_novo	Tabela de log (Etapa 2). BCNF.

Não há dependências parciais nas relações com chave composta, nem dependências transitivas nas de chave simples. As chaves candidatas foram tratadas como restrições UNIQUE.

A6–A8 — Implementação, carga e consultas

Artefato	Conteúdo	Observações
01_ddl.sql	Cria o banco cinecontrol (utf8mb4) e 13 tabelas; todas as restrições com prefixos pk_, uq_, fk_, ck_; 5 índices idx_; dois gatilhos para RN15	DROP DATABASE IF EXISTS permite reexecução em base limpa. Comentários ligam os comandos às RNs.
02_carga.sql	39 filmes, 40 usuários (35 clientes + 5 administradores), 40 atores, 10 gêneros, 10 salas, 200 assentos, 120 sessões e 200 reservas; histórico com mais de um evento por filme	Dados fictícios (e-mails @cinecontrol.local, hashes de exemplo). Contagens conforme relatório da Etapa 1. Casos de contorno: descrições nulas, períodos em aberto, filme com continuação.
03_consultas.sql	15 consultas comentadas: 5 básicas, 5 de junção/agregação e 5 avançadas	Contém consulta de três tabelas, LEFT JOIN, GROUP BY/HAVING, EXISTS e subconsultas.

Cobertura das 15 consultas: **básicas** (01–05) — WHERE, ORDER BY, BETWEEN, LIKE, IN e IS NULL; **junção e agregação** (06–10) — três tabelas com LEFT JOIN (06, 09, 10), LEFT JOIN com contagem (07) e GROUP BY/HAVING (08); **avançadas** (11–15) — subconsulta em HAVING (11 e 13), EXISTS correlacionado (12), contagem de filmes distintos por cliente (14) e ranking de salas por disponibilidade (15). A pergunta de negócio precede cada consulta no arquivo.

PARTE II

Etapa 2 — Operação, Interface e Desempenho

Na Etapa 2 o banco entra em operação. O esquema físico da Etapa 1 foi preservado; o script `04_etapa2.sql` acrescenta a camada operacional (views, auditoria, índices e perfis) e a aplicação Streamlit exercita o esquema por meio de consultas parametrizadas e transações explícitas.

Camada	Componente	Função
Banco	MySQL 8.0+ · schema cinecontrol	Tabelas da Etapa 1 + vw_relatorio_sessoes, vw_filmes_publico, auditoria_reserva, gatilhos e perfis
Acesso a dados	db.py	get_connection(), execute_query() e execute_transaction() — única porta de entrada ao MySQL
Aplicação	app.py (Streamlit)	8 páginas: painel, CRUDs, consultas, segurança e demonstração de transação
Configuração	.env (fora do Git)	DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, SENHA_SAL

B1 — Aplicação com interface

Requisito	Onde está	Situação
CRUD de ≥ 4 entidades	Filmes, Sessões e Clientes: listar, cadastrar, editar, excluir. Reservas: listar, nova reserva (transação) e cancelar (atualiza situação).	Atendido. Reservas usa cancelamento lógico em vez de exclusão, preservando o histórico e a auditoria.
2 consultas parametrizadas com junção e agregação	Página Consultas: (1) sessões por título — LIKE parametrizado com 3 tabelas; (2) ranking de filmes por ingressos e receita em um período — JOIN, GROUP BY e HAVING.	Atendido.
Relatório consolidado por view	Página Início/Relatório consulta vw_relatorio_sessoes.	Atendido.
Operação em ≥ 2 tabelas, em uma transação	Nova reserva (INSERT em reserva + UPDATE em sessao) e cancelamento (UPDATE em reserva + UPDATE em sessao); cadastro de cliente (INSERT em usuario + cliente).	Atendido.
Tratamento de erro de restrição	Os blocos try/except capturam mysql.connector.Error e exibem e.msg (mensagem do servidor) com prefixo “Restrição:” ou “Transação revertida:”, sem a exceção bruta do driver.	Atendido; a mensagem vem do MySQL e pode ser refinada para texto de domínio.

Consultas parametrizadas da interface

```
-- Consulta 1: sessões por filme (filtro por título)
SELECT f.titulo, f.classificacao, sa.nome_numero AS sala,
       s.inicio, s.preco, s.ingressos_disponiveis, s.situacao
FROM sessao s
JOIN filme f  ON f.id_filme = s.id_filme
JOIN sala sa ON sa.id_sala = s.id_sala
WHERE f.titulo LIKE %s          -- parâmetro: '%' + filtro + '%'
ORDER BY s.inicio;

-- Consulta 2: ranking de filmes por período
SELECT f.titulo, f.classificacao,
       COUNT(DISTINCT r.id_reserva) AS qtd_reservas,
       SUM(r.quantidade)          AS ingressos_vendidos,
       SUM(r.quantidade * s.preco) AS receita_total
FROM reserva r
JOIN sessao s ON s.id_sessao = r.id_sessao
JOIN filme  f ON f.id_filme  = s.id_filme
WHERE r.situacao = 'CONFIRMADA'
  AND DATE(r.data_reserva) BETWEEN %s AND %s-- parâmetros: data inicial e final
GROUP BY f.id_filme, f.titulo, f.classificacao
HAVING  SUM(r.quantidade) > 0
ORDER BY ingressos_vendidos DESC;
```

Relatório consolidado — view vw_relatorio_sessoes

```
CREATE OR REPLACE VIEW vw_relatorio_sessoes AS
SELECT f.titulo AS filme, f.classificacao, s.inicio AS horario, sa.nome_numero AS sala,
       s.preco, s.ingressos_disponiveis, s.situacao AS situacao_sessao,
       COUNT(r.id_reserva) AS total_reservas,
       COALESCE(SUM(r.quantidade), 0) AS ingressos_vendidos
FROM sessao s
JOIN filme f  ON f.id_filme = s.id_filme
JOIN sala sa ON sa.id_sala = s.id_sala
LEFT JOIN reserva r ON r.id_sessao = s.id_sessao AND r.situacao = 'CONFIRMADA'
GROUP BY s.id_sessao, f.titulo, f.classificacao, s.inicio, sa.nome_numero,
         s.preco, s.ingressos_disponiveis, s.situacao;
```

B2 — Camada de acesso a dados

- **Ponto único de acesso.** Todo SQL passa por `execute_query(sql, params)` e `execute_transaction(operations)`, em db.py. Os valores nunca são concatenados ao texto SQL: viajam separados, em tupla, ligados aos marcadores `%s` pelo driver mysql-connector-python.
- **Conexão com autocommit desligado.** Cada chamada abre uma conexão, executa, confirma ou reverte e fecha (finally). Em caso de erro, execute_query e execute_transaction chamam rollback() antes de relançar a exceção.
- **Credenciais fora do código.** Host, porta, usuário, senha e nome do banco vêm de variáveis de ambiente ou do arquivo `.env`, carregado por python-dotenv. O .gitignore exclui `.env`, `.env.*` e `**/.env`, e o arquivo não consta nos arquivos versionados do repositório.

Tentativa de injeção de SQL

A página **Segurança (demo)** envia ao campo de busca por título a entrada clássica de injeção. A aplicação a trata como dado, não como código:

```
Entrada usada:    ' OR '1'='1

Consulta executada (valor vai como parâmetro, não no texto SQL):
  SELECT id_filme, titulo FROM filme WHERE titulo LIKE %s LIMIT 5
  parâmetros = ("%' OR '1'='1%",)

O MySQL compara o título com o texto literal  %' OR '1'='1%.
Nenhum título contém essa sequência, logo o resultado esperado é 0 filmes.
Se a consulta fosse concatenada, a condição viraria sempre verdadeira e retornaria filmes.
```

B3 — Transações e controle de concorrência

Operação transacional de negócio: reservar ingressos

A reserva altera duas tabelas. As duas instruções precisam acontecer juntas ou não acontecer (RN07). No código, execute_transaction chama `start_transaction()`, executa as operações em ordem e finaliza com `commit()`; qualquer erro dispara `rollback()`.

```
-- SUCESSO
START TRANSACTION;
INSERT INTO reserva (id_cliente, id_sessao, quantidade) VALUES (1, 1, 2); -- dispara a auditoria
UPDATE sessao SET ingressos_disponiveis = ingressos_disponiveis - 2
 WHERE id_sessao = 1;                                                    -- RN07
COMMIT;

-- FALHA PROVOCADA NO MEIO DA TRANSAÇÃO
START TRANSACTION;
INSERT INTO reserva (id_cliente, id_sessao, quantidade) VALUES (1, 1, 1);    -- passo 1: válido
-- passo 2: viola ck_sessao_disponibilidade
UPDATE sessao SET ingressos_disponiveis = -9999 WHERE id_sessao = 1;
-- ERROR 3819: Check constraint 'ck_sessao_disponibilidade' is violated.
ROLLBACK; -- a reserva do passo 1 desaparece
```

Resultado esperado nos dois desfechos. Com COMMIT: uma linha nova em reserva, saldo da sessão reduzido em 2 e uma linha INSERT em auditoria_reserva. Com ROLLBACK: nenhuma linha nova em reserva, saldo da sessão inalterado e nenhuma linha nova na auditoria — o gatilho roda dentro da mesma transação, e seu registro também é desfeito. A página **Transação (demo)** executa os dois cenários com botões distintos.

Experimento de concorrência com duas sessões

Roteiro para duas sessões de terminal do cliente `mysql`, ambas na mesma sessão de cinema (id_sessao = 1) com saldo conhecido. Primeiro, registrar o nível de isolamento em vigor em cada sessão:

```
SELECT @@transaction_isolation;    -- padrão do InnoDB: REPEATABLE-READ
```

Passo	Sessão A	Sessão B
1	`START TRANSACTION;`	`START TRANSACTION;`
2	`UPDATE sessao SET ingressos_disponiveis = ingressos_disponiveis - 2 WHERE id_sessao = 1;`	–
3	–	`UPDATE sessao SET ingressos_disponiveis = ingressos_disponiveis - 1 WHERE id_sessao = 1;` **(aguarda o bloqueio de A)**
4	`COMMIT;`	(a instrução do passo 3 é liberada) `COMMIT;`
5	`SELECT ingressos_disponiveis FROM sessao WHERE id_sessao = 1;`	mesmo SELECT: saldo reduzido em 3 no total

Comportamento esperado. O InnoDB aplica bloqueio de linha exclusivo no UPDATE de A; o UPDATE de B espera (bloqueio, não erro) até o COMMIT de A, ou até estourar `innodb_lock_wait_timeout` (padrão 50 s). Como o UPDATE usa leitura atual da linha, B parte do saldo já decrementado por A, e nenhuma atualização se perde. Para o cenário de **deadlock**, inverter a ordem de acesso: A atualiza sessao e depois reserva; B atualiza reserva e depois sessao, ambos sem COMMIT — o InnoDB aborta uma das transações com ERROR 1213 (40001) e a aplicação deve repetir a operação.

Justificativa do nível de isolamento

Adotou-se o padrão do InnoDB, **REPEATABLE READ**. A operação crítica, o decremento do saldo, é um UPDATE atômico que lê e grava a linha sob bloqueio exclusivo, e o CHECK *ck_sessao_disponibilidade* impede saldo negativo mesmo sob corrida: duas reservas simultâneas jamais vendem além do saldo. READ COMMITTED reduziria bloqueios de intervalo (gap locks), mas não é necessário para uma tabela acessada por chave primária; SERIALIZABLE acrescentaria bloqueios em leituras simples sem ganho de correção aqui. A validação "quantidade ≤ disponível" feita na interface antes da transação é apenas uma checagem de usabilidade e lê um valor possivelmente desatualizado; a garantia real é o CHECK dentro da transação.

B4 — Otimização de consultas

Três consultas críticas foram escolhidas por serem executadas a cada acesso à programação ou ao histórico do cliente. O estudo usa o arquivo `05_otimizacao.sql`. Os índices do script físico da Etapa 1 já existem (`idx_sessao_sala_inicio`, `idx_reserva_cliente_data`, `idx_filme_titulo`); os da Etapa 2 estendem esses índices com a coluna de situação.

> **Como obter um "antes" válido.**
> Os índices novos são criados em 04_etapa2.sql, antes do EXPLAIN. Para medir o antes, torne-os invisíveis ao otimizador: `ALTER TABLE sessao ALTER INDEX idx_sessao_sala_inicio_sit INVISIBLE;` (recurso do MySQL 8.0), execute o EXPLAIN e depois volte a `VISIBLE`. Para medir o ganho sobre a Etapa 1, torne invisíveis também os índices originais. Registre o tempo com `EXPLAIN ANALYZE` (8.0.18+) e informe o volume de linhas.

Consulta crítica 1 — sessões programadas de uma sala

Pergunta de negócio: Quais sessões estão programadas para uma sala específica?

```
SELECT s.id_sessao, f.titulo, s.inicio, s.ingressos_disponiveis
FROM sessao s JOIN filme f ON f.id_filme = s.id_filme
WHERE s.id_sala = 1 AND s.situacao = 'PROGRAMADA'
ORDER BY s.inicio;
```

Índice: `idx_sessao_sala_inicio_sit ON sessao (id_sala, inicio, situacao)`

Escolha e ordem das colunas. id_sala é igualdade e vem primeiro; inicio vem em seguida porque o ORDER BY pode ser atendido na ordem do índice, sem ordenação extra (Using filesort). situacao fica por último: o filtro é aplicado sobre a própria folha do índice, sem ler a linha da tabela, mas não reduz a faixa varrida. Uma alternativa a testar é (id_sala, situacao, inicio): as duas igualdades delimitam a faixa e o ORDER BY continua atendido.

Leitura esperada do plano. Sem índice em id_sala, o plano esperado é type=ALL em sessao com Using filesort. Com índice, type=ref, key=o índice, sem filesort. Como a Etapa 1 já oferece (id_sala, inicio), o ganho do índice novo sobre ele tende a ser pequeno, limitado a filtrar situacao dentro do índice.

	Antes (índice invisível)	Depois (índice visível)
type / key	a registrar	a registrar
rows estimadas	a registrar	a registrar
Extra	a registrar	a registrar
Tempo (EXPLAIN ANALYZE) e volume	a registrar	a registrar

Consulta crítica 2 — reservas confirmadas de um cliente em um período

Pergunta de negócio: Quais reservas confirmadas um cliente fez em um período?

```
SELECT r.id_reserva, f.titulo, s.inicio, r.quantidade, r.data_reserva
FROM reserva r
JOIN sessao s ON s.id_sessao = r.id_sessao
JOIN filme f ON f.id_filme = s.id_filme
WHERE r.id_cliente = 1 AND r.situacao = 'CONFIRMADA'
  AND r.data_reserva BETWEEN '2026-01-01' AND '2026-12-31'
ORDER BY r.data_reserva DESC;
```

Índice: `idx_reserva_cli_data_sit ON reserva (id_cliente, data_reserva, situacao)`

Escolha e ordem das colunas. Igualdade em id_cliente, faixa em data_reserva e ordenação pela mesma coluna: o índice entrega as linhas já ordenadas (leitura reversa para DESC). situacao, depois da faixa, só filtra dentro do índice. Os JOINs por PK (sessao e filme) usam eq_ref.

Leitura esperada do plano. Sem índice, type=ALL em reserva com filesort. Com o índice, type=range. Em relação ao idx_reserva_cliente_data da Etapa 1, o acréscimo de situacao evita leituras da tabela para linhas canceladas; o ganho depende da proporção de canceladas.

	Antes (índice invisível)	Depois (índice visível)
type / key	a registrar	a registrar
rows estimadas	a registrar	a registrar
Extra	a registrar	a registrar
Tempo (EXPLAIN ANALYZE) e volume	a registrar	a registrar

Consulta crítica 3 — filmes em cartaz por prefixo de título

Pergunta de negócio: Quais filmes em cartaz começam com determinada palavra?

```
SELECT id_filme, titulo, duracao_min, classificacao
FROM filme
WHERE situacao = 'EM_CARTAZ' AND titulo LIKE 'Horizonte%';
```

Índice: `idx_filme_titulo_sit ON filme (situacao, titulo)`

Escolha e ordem das colunas. Igualdade em situacao e faixa por prefixo em titulo: o índice permite posicionar-se diretamente no intervalo. LIKE 'X%' usa o índice; LIKE '%X%' (a consulta 1 da interface) não pode usá-lo, pois não há prefixo para a busca na árvore B.

Leitura esperada do plano. Com poucas dezenas de filmes, o otimizador pode preferir varredura completa mesmo com o índice, porque ler uma única página é mais barato que descer na árvore. Nesse volume um resultado negativo é esperado e deve ser relatado como tal; para evidenciar o ganho, repetir a medição com a tabela ampliada (por exemplo, 100 mil filmes sintéticos).

	Antes (índice invisível)	Depois (índice visível)
type / key	a registrar	a registrar
rows estimadas	a registrar	a registrar
Extra	a registrar	a registrar
Tempo (EXPLAIN ANALYZE) e volume	a registrar	a registrar

B5 — Segurança e controle de acesso

Perfis de acesso (menor privilégio)

Perfil	Uso	Privilégios concedidos em 04_etapa2.sql
cc_leitura	Acesso público, sem autenticação de aplicação	SELECT apenas nas views vw_filmes_publico e vw_relatorio_sessoes. Não enxerga tabelas-base (usuario, senha_hash, reserva).
cc_operacional	Atendente de bilheteria	SELECT em todo o schema; INSERT e UPDATE em reserva; UPDATE em sessao. Sem DELETE e sem escrita em usuario ou filme.
cc_gestor	Gestão do cinema	SELECT, INSERT, UPDATE e DELETE em todo o schema. Sem DDL (CREATE, ALTER, DROP) e sem GRANT.

```
CREATE USER IF NOT EXISTS 'cc_leitura'@'localhost' IDENTIFIED BY '<senha>';
GRANT SELECT ON cinecontrol.vw_filmes_publico TO 'cc_leitura'@'localhost';
GRANT SELECT ON cinecontrol.vw_relatorio_sessoes TO 'cc_leitura'@'localhost';

CREATE USER IF NOT EXISTS 'cc_operacional'@'localhost' IDENTIFIED BY '<senha>';
GRANT SELECT ON cinecontrol.*            TO 'cc_operacional'@'localhost';
GRANT INSERT, UPDATE ON cinecontrol.reserva TO 'cc_operacional'@'localhost';
GRANT UPDATE ON cinecontrol.sessao      TO 'cc_operacional'@'localhost';

CREATE USER IF NOT EXISTS 'cc_gestor'@'localhost' IDENTIFIED BY '<senha>';
GRANT SELECT, INSERT, UPDATE, DELETE ON cinecontrol.* TO 'cc_gestor'@'localhost';

-- REVOKE: retirada de um privilégio quando o perfil deixa de precisar dele
REVOKE UPDATE ON cinecontrol.sessao FROM 'cc_operacional'@'localhost';
```

Visões de restrição

vw_filmes_publico expõe apenas id_filme, titulo, duracao_min, classificacao e data_lancamento dos filmes com situacao = 'EM_CARTAZ': restringe colunas (omite descrição e situação) e linhas (omite filmes futuros, pausados e encerrados). **vw_relatorio_sessoes** entrega agregados de sessão sem expor dados de clientes.

Armazenamento de senhas

A senha nunca é gravada em texto claro. No cadastro de cliente, `hash_senha()` calcula `SHA-256(sal + senha)`, com o sal lido da variável SENHA_SAL (ambiente), e grava o resultado em usuario.senha_hash. A página Segurança (demo) mostra o hash gerado para uma senha digitada.

> **Limitação reconhecida.**
> O sal é um valor único para toda a aplicação, e não um sal aleatório por usuário, e o SHA-256 é uma função rápida. Isso atende ao requisito de hash com sal do roteiro, mas em produção a recomendação é bcrypt, scrypt ou argon2, que embutem sal aleatório por senha e custo configurável. Esta troca está listada como evolução em B7. As senhas dos perfis de banco, presentes em 04_etapa2.sql, são de demonstração e devem ser substituídas em qualquer ambiente real.

Dados pessoais e cuidados adotados

Dado	Tabela	Classificação	Cuidado adotado
Nome e e-mail do usuário	usuario	Pessoal (identifica o titular)	Cargas com dados fictícios (domínio @cinecontrol.local); perfil público não acessa a tabela; e-mail único e validado por UNIQUE.
Senha	usuario.senha_hash	Pessoal e sensível (credencial)	Somente hash com sal; nunca exibida; SENHA_SAL fora do código.
Histórico de reservas	reserva, auditoria_reserva	Pessoal (hábitos de consumo, vinculados ao cliente)	Acesso restrito aos perfis operacional e gestor; auditoria sem dados cadastrais, apenas ids e valores da reserva.
Data de nascimento	ator	Pessoal, mas de personagens públicos fictícios	Opcional (aceita NULL); dados sintéticos.

Princípios aplicados (LGPD): **minimização** (o modelo não coleta CPF, telefone nem endereço), **finalidade** (cada dado serve ao acesso ou à reserva) e **segurança** (menor privilégio, hash de senha e credenciais fora do repositório). Nenhum dado real de terceiros foi usado na carga.

B6 — Gatilhos e auditoria

O que auditar e por quê. A reserva é o evento central do domínio e a que gera disputa (ingresso vendido, cancelado ou alterado). Por isso a auditoria cobre as **inserções** de reserva e as **alterações de situação ou quantidade** (cancelamentos). A tabela `auditoria_reserva` guarda operação, id da reserva, autor (`USER()`, conta do banco), data e hora e os valores anterior e posterior em JSON.

```
CREATE TABLE IF NOT EXISTS auditoria_reserva (
    id_auditoria BIGINT AUTO_INCREMENT,
    operacao     VARCHAR(10)  NOT NULL,                    -- INSERT | UPDATE
    id_reserva   BIGINT,
    autor        VARCHAR(100) DEFAULT (USER()),
    momento      TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    valor_ant    JSON,
    valor_novo   JSON,
    CONSTRAINT pk_auditoria PRIMARY KEY (id_auditoria)
) ENGINE=InnoDB;

CREATE TRIGGER trg_audit_reserva_ins AFTER INSERT ON reserva FOR EACH ROW
  INSERT INTO auditoria_reserva (operacao, id_reserva, valor_novo)
  VALUES ('INSERT', NEW.id_reserva,
          JSON_OBJECT('id_cliente', NEW.id_cliente, 'id_sessao', NEW.id_sessao,
                      'quantidade', NEW.quantidade, 'situacao', NEW.situacao));

CREATE TRIGGER trg_audit_reserva_upd AFTER UPDATE ON reserva FOR EACH ROW
BEGIN
  IF OLD.situacao <> NEW.situacao OR OLD.quantidade <> NEW.quantidade THEN
    INSERT INTO auditoria_reserva (operacao, id_reserva, valor_ant, valor_novo)
    VALUES ('UPDATE', NEW.id_reserva,
            JSON_OBJECT('situacao', OLD.situacao, 'quantidade', OLD.quantidade),
            JSON_OBJECT('situacao', NEW.situacao, 'quantidade', NEW.quantidade));
  END IF;
END
```

Escolhas de projeto. O gatilho de UPDATE só registra quando situação ou quantidade mudam, para não poluir a trilha com alterações sem significado de negócio. id_reserva não tem chave estrangeira, para que o rastro sobreviva a eventual exclusão da reserva. Por serem AFTER, os gatilhos participam da transação da reserva: se a operação for revertida, o registro de auditoria também é. Como a aplicação conecta com uma única conta de banco, o campo autor identifica essa conta, e não o usuário final do sistema; propagar o usuário da aplicação é uma evolução prevista.

B7 — Projeto físico e armazenamento

Revisão dos tipos de dados

Atributo	Tipo atual	O que mudaria e por quê
sessao.inicio, historico.inicio/fim	TIMESTAMP	DATETIME. TIMESTAMP tem limite em 2038 e converte pelo fuso da sessão; o horário de uma sessão é um instante local de calendário. Mantém-se TIMESTAMP em reserva.data_reserva e auditoria.momento, que registram o instante em que algo ocorreu.
Chaves BIGINT em tabelas pequenas	BIGINT	INT UNSIGNED em usuario, filme, genero, ator, sala e sessao (até 4 bilhões); BIGINT permanece em reserva e auditoria_reserva, de maior crescimento. Economiza 4 bytes por linha em cada tabela e em cada FK e índice que a referencia.
sessao.ingressos_disponiveis, sala.capacidade	INTEGER	SMALLINT UNSIGNED seria suficiente (capacidade ≤ 1000). Não foi aplicado nas colunas com CHECK >= 0, pois UNSIGNED trocaria o erro de CHECK por erro de faixa, menos legível na interface.
situacao, classificacao, nivel_acesso	VARCHAR + CHECK	Mantido: o CHECK é legível e portátil. ENUM ou tabela de domínio seriam alternativas se os valores passassem a ser editáveis pelo negócio.
usuario.senha_hash	VARCHAR(255)	Mantido; comporta hashes de bcrypt (60) e argon2 (~100) quando a função for trocada (B5).
sessao.preco	NUMERIC(8,2)	Mantido: tipo exato para valor monetário; nunca FLOAT.

Estimativa de crescimento (horizonte de 2 anos)

Premissas explícitas, a serem ajustadas ao cinema real: 10 salas; 6 sessões por sala por dia; 365 dias; média de 25 reservas por sessão; 15% das reservas canceladas depois de feitas; tamanho médio de linha estimado em ~110 bytes para reserva (dados + índices) e ~250 bytes para auditoria_reserva (JSON).

Tabela	Cálculo	Linhas em 2 anos	Tamanho estimado
sessao	10 × 6 × 365 × 2	≈ 43.800	≈ 6 MB
reserva	43.800 × 25	≈ 1.095.000	≈ 120 MB
auditoria_reserva	1 INSERT por reserva + 15% de UPDATE de cancelamento	≈ 1.260.000	≈ 315 MB
usuario / cliente	Crescimento orgânico	milhares	desprezível

São estimativas por premissa, não medições. Para validá-las, comparar com `information_schema.TABLES` (data_length + index_length) após uma carga proporcional.

Plano de backup

Item	Definição
O que copiar	Banco cinecontrol completo (dados, views, gatilhos, rotinas) e os privilégios dos perfis cc_* (SHOW GRANTS), mais os binary logs.
Estratégia	Cópia lógica completa diária com `mysqldump --single-transaction --routines --triggers --events cinecontrol` (consistente para InnoDB sem bloquear leituras) + binary logs ativados para recuperação até um ponto no tempo (PITR).
Frequência	Completo diário às 02:00 (janela de baixa movimentação); binary logs contínuos.
Retenção	7 diários, 4 semanais e 6 mensais; binary logs por 7 dias (binlog_expire_logs_seconds). Cópias em volume distinto do servidor, com verificação de checksum.
Teste de restauração	Mensal, em instância descartável: restaurar o último dump, aplicar binlogs até um horário-alvo, comparar contagem de linhas por tabela com a origem e executar 03_consultas.sql. Backup sem restauração testada não é considerado válido.

Particionamento e arquivamento histórico

Com ~1,1 milhão de reservas e ~1,3 milhão de registros de auditoria em dois anos, nenhuma tabela exige particionamento hoje. A proposta é **arquivar**, e não particionar: mover reservas de sessões encerradas há mais de 12 meses e a auditoria correspondente para tabelas de arquivo (reserva_arquivo, auditoria_reserva_arquivo) em rotina mensal, mantendo a base operacional enxuta. Se o volume crescer a ponto de justificar partições, **auditoria_reserva** é a candidata natural, por RANGE sobre *momento* com partições mensais; o MySQL exige que a coluna de particionamento faça parte de toda chave primária e única, o que pediria uma PK composta (id_auditoria, momento) e a remoção de chaves estrangeiras (a tabela já não as tem).

Evoluções previstas

- Trocar SHA-256 com sal global por bcrypt ou argon2 com sal por usuário (B5).
- Propagar o usuário da aplicação para a auditoria (por exemplo, variável de sessão @usuario_app lida pelos gatilhos).
- Tela de login e verificação de titularidade no cancelamento (RN01, RN02 e RN09).
- Rejeitar sobreposição de intervalos de sessão na mesma sala (RN06 hoje cobre início idêntico), com gatilho que considere a duração do filme.

B8 — Revisão da Etapa 1 e relatório final

O roteiro exige que o relatório final incorpore as correções apontadas na devolutiva da Etapa 1. As mudanças abaixo foram identificadas no histórico do repositório e no confronto entre o relatório da Etapa 1 e o código atual.

Item	Etapa 1 (entrega de 20/09/2026)	Revisão neste relatório
Modelo lógico	Entregue em docs/modelo-logico.pdf	Corrigido em 29/09/2026 (commit “fix-modelo_logico.pdf”) e reproduzido na Figura 2.
Decisões de mapeamento	Descritas de forma resumida	Justificadas em texto, uma a uma, com a alternativa descartada (A4).
Normalização	Afirmava ausência de desnormalização	Passa a declarar as redundâncias controladas R1 e R2, a justificativa e a análise BCNF (A5).
Rastreabilidade das RNs	Indicava apenas o tipo de atendimento	Aponta o objeto físico (restrição, gatilho, consulta ou trecho da aplicação) de cada RN (A1).
Dicionário de dados	Referenciado em PDF separado	Versão condensada incluída, com a tabela de auditoria da Etapa 2 (A3).
Camada operacional	Inexistente	Views, auditoria, índices, perfis e aplicação (Parte II).

Declaração de uso de IA

Em conformidade com o item 9 do roteiro, a equipe declara o uso de ferramentas de inteligência artificial como apoio:

Parte do trabalho	Uso da IA	Verificação pela equipe
Etapa 1 — documentação e modelo	Organização do relatório, proposta de ampliação do modelo (gênero, elenco, assento, histórico, especialização) e estruturação dos artefatos.	Conferência do MER, do dicionário e do modelo lógico contra o script 01_ddl.sql; execução dos scripts em base limpa.
Etapa 2 — código	Apoio à escrita de app.py, db.py e dos scripts 04_etapa2.sql, 05_otimizacao.sql e seed_etapa2.sql (o histórico do repositório indica uso de assistente de código).	Execução da aplicação e dos scripts em MySQL 8.0, teste dos cenários de COMMIT e ROLLBACK e revisão do código por todos os integrantes.
Este relatório final	Redação e diagramação do relatório a partir do repositório, com o assistente Claude.	Leitura integral e conferência dos números e comandos contra os arquivos do repositório; preenchimento das evidências marcadas “a anexar”.

Reprodução do ambiente

```
# Etapa 1 – banco do zero (a partir da raiz do repositório)
mysql -u root -p < qsl/01_ddl.sql
mysql -u root -p cinecontrol < qsl/02_carga.sql
mysql -u root -p cinecontrol < qsl/03_consultas.sql

# Etapa 2 – camada operacional e dados adicionais (opcional)
mysql -u root -p cinecontrol < cinecontrol_etapa2/app/sql/04_etapa2.sql
# CUIDADO: o seed executa TRUNCATE em todas as tabelas
mysql -u root -p cinecontrol < cinecontrol_etapa2/app/sql/seed_etapa2.sql

# Aplicação
cd cinecontrol_etapa2/app
pip install -r requirements.txt
# criar .env com DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, SENHA_SAL
streamlit run app.py          # http://localhost:8501
```

Atenção: `01_ddl.sql` recria o banco e apaga dados anteriores, e `seed_etapa2.sql` executa TRUNCATE nas tabelas, inclusive em auditoria_reserva, para recomeçar a partir de uma carga mais rica.

Anexo C — Checklist da Etapa 2

Item do checklist	Situação	Comentário
A aplicação sobe em máquina limpa seguindo o README	**Evidência a anexar**	Instruções no README e neste relatório; testar em máquina sem o ambiente atual.
Os quatro CRUDs, as duas consultas e o relatório funcionam	**Atendido**	Reservas sem exclusão física (cancelamento lógico).
Nenhuma consulta é montada por concatenação de string	**Atendido**	Todas usam %s com tupla de parâmetros; o texto SQL é fixo.
As credenciais do banco não estão versionadas	**Atendido**	.env ignorado pelo Git e ausente dos arquivos versionados.
Operação transacional testada em sucesso e falha	**Evidência a anexar**	Código e roteiro prontos; anexar as saídas (B3).
Experimento de concorrência documentado	**Evidência a anexar**	Roteiro descrito; registrar o observado (B3).
Três estudos de EXPLAIN com plano antes, índice, plano depois e tempo	**Evidência a anexar**	Estudos desenhados; medir e preencher as tabelas (B4).
Três perfis criados e testados com login real	**Evidência a anexar**	Perfis criados em 04_etapa2.sql; anexar os testes (B5).
Senhas armazenadas com hash e sal	**Parcial**	SHA-256 com sal global; evolução para bcrypt/argon2 prevista (B5).
Dois gatilhos de auditoria com autor, data e valores	**Atendido**	Autor é a conta do banco (B6).
Plano de backup com frequência e retenção	**Atendido**	B7.
Correções da devolutiva da Etapa 1 incorporadas	**Evidência a anexar**	Preencher os apontamentos recebidos (B8).
Ambiente de demonstração preparado	**Evidência a anexar**	Banco povoado e duas sessões de terminal abertas antes da apresentação.
Seção de declaração de uso de IA preenchida	**Atendido**	Revisar com a equipe.